

Process Scheduling

Part 2

Sistemas de Computadores

Departamento de Engenharia Informática

Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Overview

- Non-preemptive scheduling
- Preemptive scheduling
- Multilevel queue scheduling
- Starvation
- Resource sharing

Priority-based scheduling

■ Process priority

- A numeric value that ranks processes based on their worth and need for processor time

■ Can either be defined:

- Internally, e.g., CPU or memory requirements, I/O, ...
- Externally, e.g., importance to the user/system, amount being paid for computation time, ...

■ The basic idea is straightforward

- Run **highest priority processes first**
- Use FIFO/RR within same priority

Static priorities

- **Predetermined and assigned fixed values at process creation that remain unchanged throughout the task's lifecycle**
- **Built on external criteria like system user role or job importance**
- + **Avoids the computational costs of dynamic re-evaluation during runtime**
- **Vulnerable to starvation if high-priority queues are continuously filled**

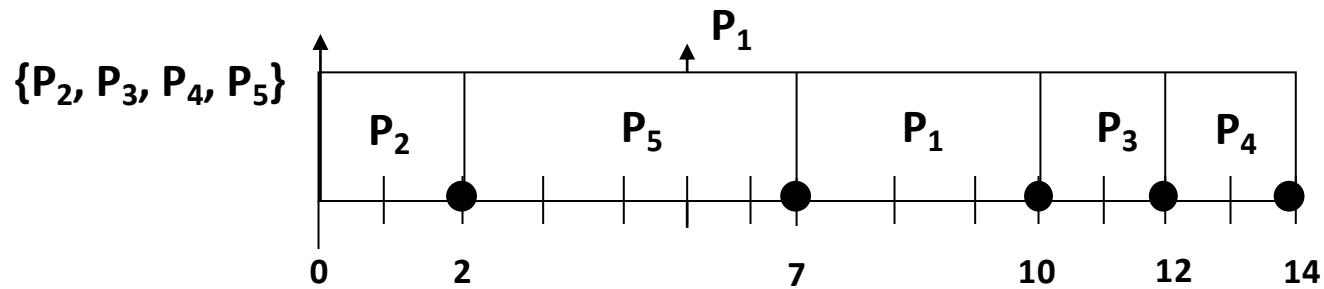
Dynamic priorities

- Adjusted responsively by the scheduler in real-time, adapting to the runtime execution behavior of processes
- Computed using internal metrics such as I/O burst history or active CPU wait time
- + Starving tasks are dynamically promoted (e.g., via "aging" concepts)
- Requires additional processor cycles to periodically recalculate rankings

Non-preemptive scheduling

- Processes are scheduled according to the priority number assigned to them
 - The highest-priority process is chosen to run
- Once a process enters the running state, it continues executing until:
 - It **completes**
 - It **blocks** (e.g., I/O request)
 - It **voluntarily yields** the CPU

Non-preemptive scheduling

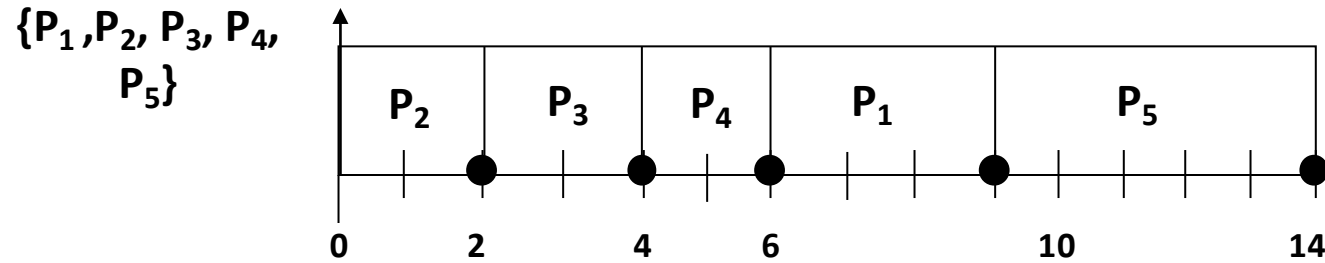


Process	Execution time	Priority
P ₁	3	2
P ₂	2	1 (highest)
P ₃	2	4
P ₄	2	5 (lowest)
P ₅	5	3

Shortest Job First

- **Improving on FIFO: shorter jobs should run first**
 - Prioritizing shorter CPU bursts reduces average waiting and turnaround time
- **Each arriving process is inserted into the ready queue based on its estimated execution time**
 - The **ready queue is kept ordered** by increasing execution time
 - When the CPU becomes available, the process with the shortest execution time is selected
- **If two processes have the same execution time, FIFO order is used**

Shortest Job First



Process	Execution time
P_1	3
P_2	2
P_3	2
P_4	2
P_5	5

Shortest Job First

+ Optimal average waiting and turnaround time

- Proven when all processes are available simultaneously
- No other scheduling algorithm achieves lower average turnaround in this scenario

– Requires knowledge (or estimation) of CPU burst time

- Exact execution times are rarely known in advance
- Typically estimated using past behavior (e.g., exponential averaging of previous CPU bursts)

– Bias against long processes

- Processes with large CPU bursts may experience long waiting times
- In heavily loaded systems, short jobs are repeatedly favored
- Can lead to starvation of long processes

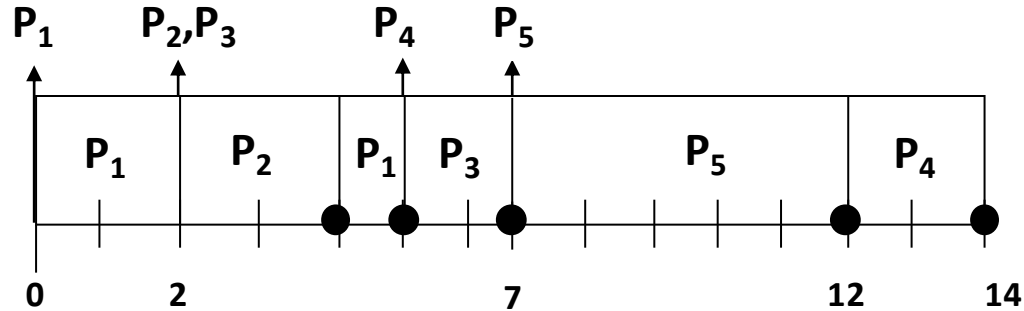
Overview

- Non-preemptive scheduling
- **Preemptive scheduling**
- Multilevel queue scheduling
- Starvation
- Resource sharing

Preemptive scheduling

- Preemption is the act of temporarily interrupting the current process, **without requiring its cooperation**
 - With the intention of resuming the process at a later time
- Upon arrival of a process in the ready queue, its priority is compared with:
 1. The priority of the other processes present in the ready queue
 2. As well as with the one which is being executed by the CPU
- CPU is always assigned to the highest-priority runnable process

Preemptive scheduling



Process	Execution time	Priority	Arrival time
P ₁	3	3	0
P ₂	2	1 (highest)	2
P ₃	2	4	2
P ₄	2	5 (lowest)	5
P ₅	5	2	7

Shortest Remaining Time First

- **A preemptive version of SJF**

- Well suited for time-sharing systems with frequent arrivals
- Improves responsiveness by favoring short remaining jobs

- **The ready queue is effectively ordered by remaining time**

- The scheduler selects the process with the **smallest remaining execution time**

- **Processes will always run until they complete, or a new process is added that requires a smaller amount of time**

- Preemption occurs when an incoming process arrives with a shorter remaining time than the remaining execution time of the active process

Shortest Remaining Time First

+ Lower average waiting time than SJF in dynamic workloads

- More responsive to newly arriving short processes
- Theoretically optimal (minimum average waiting/turnaround time), assuming an accurate knowledge of remaining execution times

– Higher overhead and implementation complexity

- Requires continuous tracking of remaining time for all processes
- Frequent preemptions lead to more context switches

– Risk of starvation

- Long processes may be delayed indefinitely if shorter jobs keep arriving

Comparison of scheduling algorithms

Algorithm	Preemptive	Strengths	Weaknesses	Risk of Starvation	Typical Use
FIFO	No	Simple, low overhead	Poor responsiveness, convoy effect	No	Batch systems
SJF	No	Optimal average turnaround (if known)	Requires burst estimation	Yes (long jobs)	Batch workloads
SRTF	Yes	Best average turnaround (dynamic)	High overhead, complex	Yes (long jobs)	Time-sharing (theoretical)
RR	Yes	Fair, good responsiveness	Context-switch overhead	No	Interactive systems

Overview

- Non-preemptive scheduling
- Preemptive scheduling
- **Multilevel queue scheduling**
- Starvation
- Resource sharing

Multilevel queue scheduling

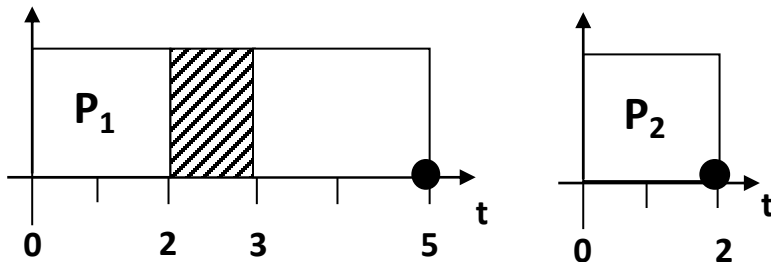
- **Used when processes can be classified into distinct groups**
 - E.g., foreground (interactive) vs background (batch)
- **The ready queue is divided into **multiple separate queues****
 - Each queue corresponds to a specific class of processes
 - Processes are **statically** assigned to a queue (no movement between queues)
- **Each queue can use a different scheduling algorithm, e.g.:**
 - Interactive processes → Round-Robin (RR)
 - Batch processes → FIFO
 - System processes → priority-based scheduling

Multilevel queue scheduling

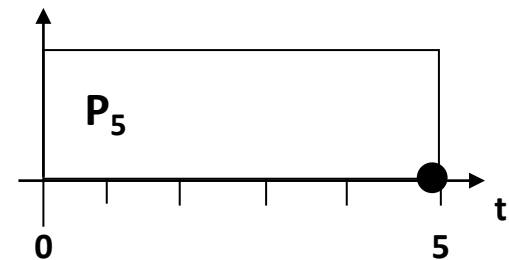
- In addition to scheduling within each queue, the OS must also decide **which queue gets access to the CPU**
 - Ensure fair CPU allocation
 - Respect relative importance of process classes
- **Option 1: Preemptive fixed-priority scheduling**
 - Each queue is assigned a priority level
 - The CPU is always given to the highest-priority non-empty queue
 - Lower-priority queues run only if higher-priority queues are empty
- **Option 2: Round-Robin among queues**
 - Each queue is assigned a time slice (quantum)
 - Queues are scheduled in circular order
 - Within its time slice, each queue schedules its processes using its own policy

Multilevel queue scheduling: Example

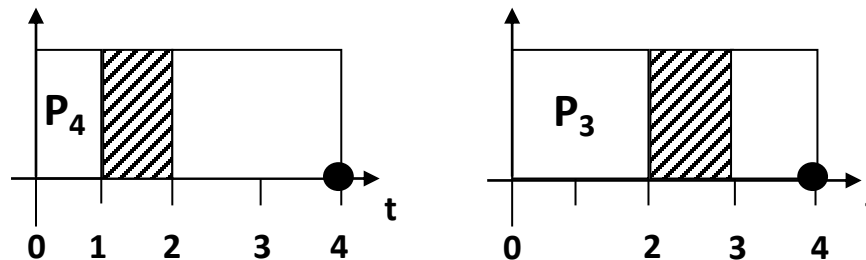
System processes



Batch processes



Interactive processes



Multilevel queue scheduling: Example

■ Per-queue schedulers

- System: preemptive fixed priority ($P_1 = 2$, $P_2 = 1$)
- Interactive: RR with a time slice of 2 (P_3 , P_4)
- Batch: FIFO (P_5)

■ Preemptive fixed priority among queues

- System = 1 (highest)
- Interactive = 2
- Batch = 3 (lowest)

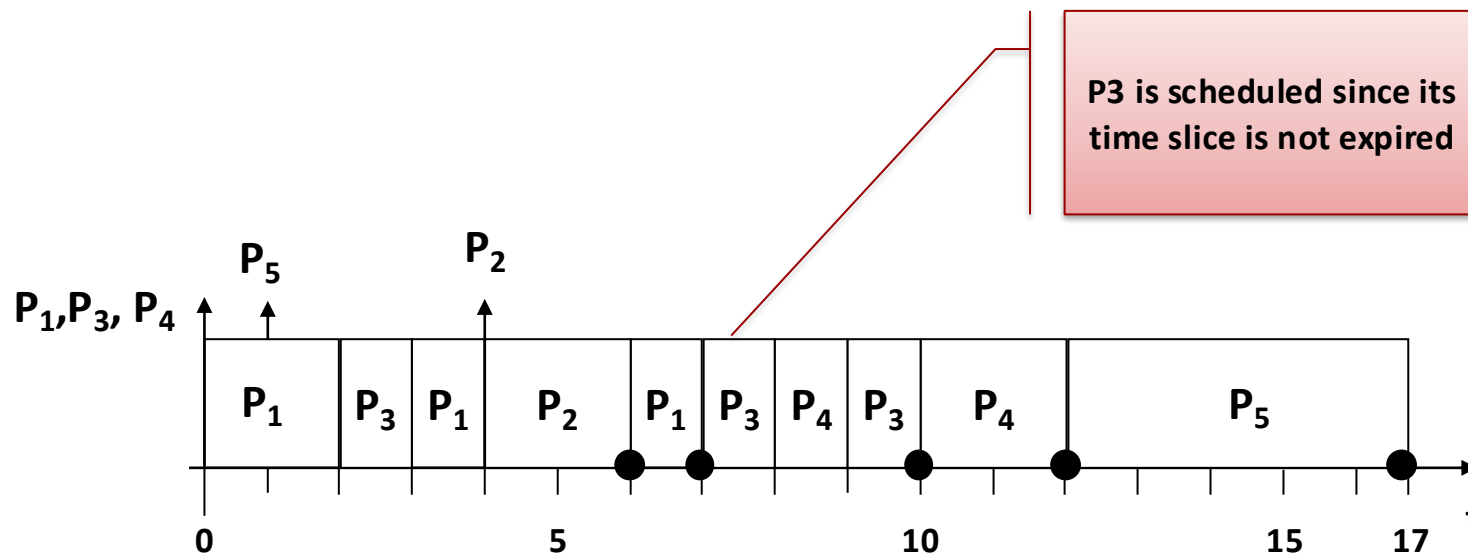
Multilevel queue scheduling: Example

■ Per-queue schedulers

- System: preemptive fixed priority ($P_1 = 2, P_2 = 1$)
- Interactive: RR with a time slice of 2 (P_3, P_4)
- Batch: FIFO (P_5)

■ Preemptive fixed priority

- System = 1 (highest)
- Interactive = 2
- Batch = 3 (lowest)



Multilevel queue scheduling

+ Simple and efficient structure

- Clear separation of process classes
- Each queue can use the most appropriate policy
- Different types of processes handled differently
- No need to dynamically move processes between queues

– Inflexible (static assignment)

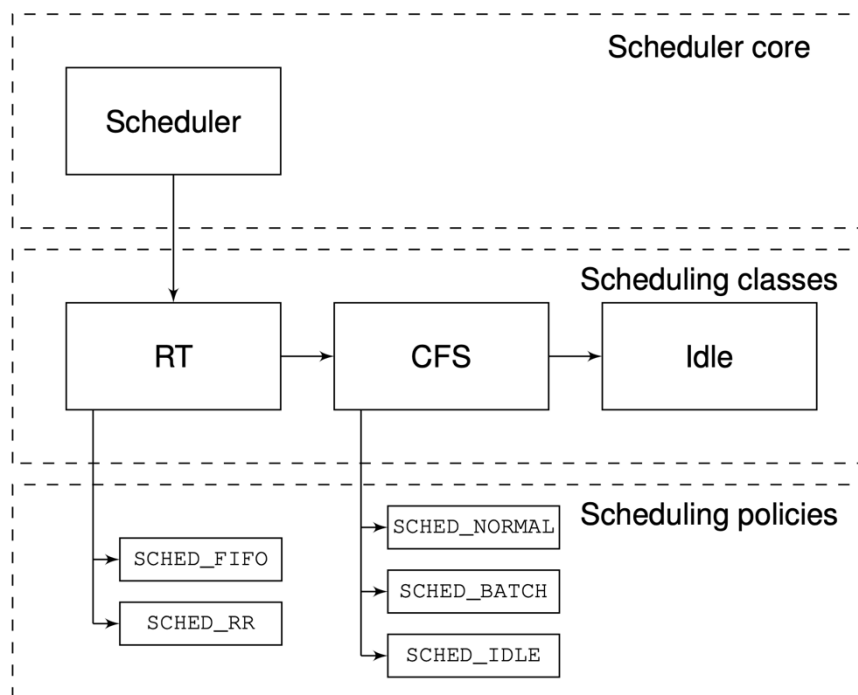
- Processes cannot move between queues, even if behavior changes
- Incorrect assignment leads to poor performance
- Does not respond well to dynamic workloads

– Risk of starvation

- Lower-priority queues may receive little or no CPU time
- Especially with fixed-priority scheduling among queues

The Linux core scheduler

- Essentially, the core scheduler is a dispatcher that interacts with two other components
 - Scheduling classes, with different scheduling policies
 - A low-level task switching mechanism



Multilevel feedback queue scheduling

- **Process behavior may not be static**

- May alternate between interactive and compute-intensive phases

- **Applications like MATLAB, Photoshop, or media tools:**

- Interactive phases (user input, UI) → short bursts
- Compute-intensive phases (processing, rendering) → long bursts

- **With MLFQ scheduling, processes can move between queues dynamically**

- Processes are classified based on their CPU burst characteristics
- Scheduling adapts to the observed behavior of each process

Multilevel feedback queue scheduling

- **Multiple queues are maintained, each with a different priority level**
 - Higher-priority queues preempt lower-priority ones
- **Queues are often scheduled using Round-Robin**
 - Shorter time slices for higher-priority queues
 - Longer time slices for lower-priority queues
- **New processes are inserted at the end of the highest-priority queue**

Multilevel feedback queue scheduling

■ Preempted processes (use full time slice):

- Moved to a lower-priority queue
- Indicates CPU-bound behavior

■ Processes that yield or block early:

- Remain in the same queue
- Indicates I/O-bound or interactive behavior

MLFQ: Adaptation and fairness

■ Favoring interactive processes

- I/O-bound and interactive processes remain in higher-priority queues
- Characterized by short CPU bursts and frequent blocking
- Ensures good responsiveness

■ Processes may be promoted to higher-priority queues

- If they frequently yield or block early (“good behavior”)
- If they wait too long in lower-priority queues (**aging**)

Multilevel feedback queue scheduling

+ Adaptive behavior

- Dynamically adjusts to process characteristics
- Handles changing workloads effectively
- Approximates SJF without knowing CPU burst times

+ Balances multiple goals

- Combines throughput, latency, and fairness
- Favors interactive and I/O-bound processes
- Short jobs are scheduled quickly

+ Reduces starvation

- Promotion/aging mechanisms ensure all processes eventually run

Multilevel feedback queue scheduling

- **More difficult to design, implement, and tune**
 - Performance depends on number of queues, time slice sizes, and promotion/demotion policies
 - Frequent priority changes and context switches
- **Less predictable behavior**
 - Behavior can be harder to analyze compared to simpler algorithms

Overview

- Non-preemptive scheduling
- Preemptive scheduling
- Multilevel queue scheduling
- **Starvation**
- Resource sharing

Starvation

- **Scheduling based solely on fixed priorities lacks inherent fairness**
 - Without mechanisms to balance resource distribution, high-priority tasks can monopolize the CPU

- **A process eligible to run may never execute, leading to indefinite postponement (starvation)**
 - Low-priority processes may **remain in the ready state indefinitely**, if higher-priority tasks are always available

- **Urban legend about the IBM 7074 at MIT**
 - When it was shut down in 1973, low-priority processes were found that had been submitted in 1967 and had not yet been run...

Starvation

- **Starvation is often a byproduct of simplistic scheduling that ignores long-term fairness**
 - Beyond efficiency, a scheduler must ensure no process is perpetually deprived of essential resources
- **Contemporary algorithms are designed to provide a “minimum execution time” for every task**
- **Primary solution: Aging**
 - Priority is no longer static; it evolves based on wait time
 - As a process lingers in the ready queue, its priority incrementally increases until execution is guaranteed

Starvation

- **Scheduling algorithms that prevent starvation are considered fair**
 - Ensure that all processes receive a reasonable share of CPU time
- **Fairness often introduces overhead**
 - Processes may execute slower than in more aggressive, less fair policies
- **Example: Lock convoy**
 - Occurs when multiple processes of equal priority contend for the same lock
 - Processes make progress (no deadlock), but at a significantly reduced rate
 - Overall system performance degradation

Linux Completely Fair Scheduler

- **The Completely Fair Scheduler (CFS) is the default process scheduler in the Linux kernel**
 - Designed to provide fair CPU allocation among processes, ensuring that each process receives a fair share of CPU time based on its weight or priority
- **Unlike traditional schedulers, CFS does not use fixed time slices**
 - Instead, it employs a concept called “virtual runtime” to dynamically prioritize processes that represents the amount of CPU time it has received relative to its weight
 - Processes with smaller virtual runtimes are prioritized for execution
- **CFS uses a red–black tree ordered by virtual runtime**

Overview

- Non-preemptive scheduling
- Preemptive scheduling
- Multilevel queue scheduling
- Starvation
- **Resource sharing**

Resource sharing and scheduling

■ Mechanisms such as locks control access to shared resources

- Ensure correct coordination of concurrent accesses
- Prevent race conditions and data corruption

■ Interaction with scheduling

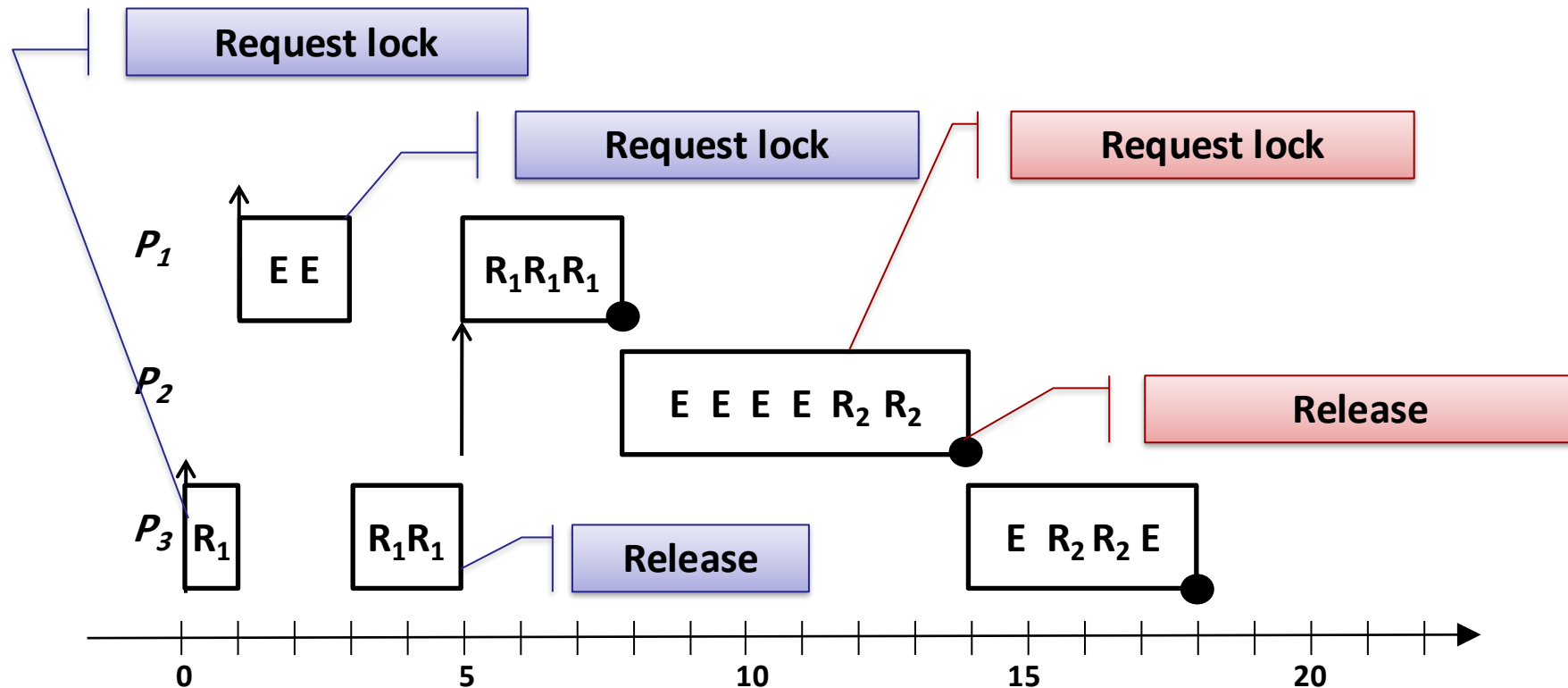
- Processes share resources beyond the CPU: memory, files, devices, network, etc.
- Scheduling decisions must consider resource dependencies

■ Key issue with priority scheduling

- Static priority scheduling is vulnerable to resource sharing effects

Resource sharing

Process	Execution time	Priority	Arrival instant	Execution sequence
P_1	5	1	1	E E $R_1 R_1 R_1$
P_2	5	2	5	E E E E $R_2 R_2$
P_3	7	3	0	$R_1 R_1 R_1$ E $R_2 R_2$ E



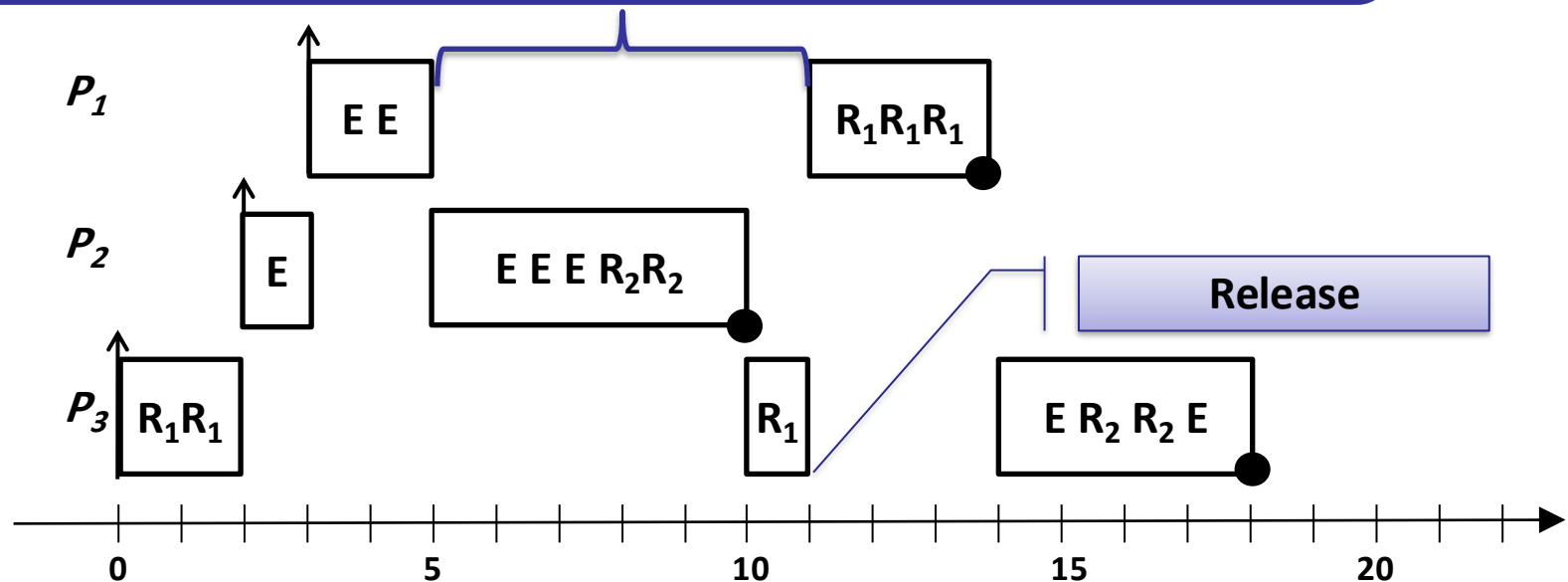
Priority inversion

- When a higher priority process is **indirectly preempted** by a lower priority one
 - Effectively “inverting” the relative priorities of the two processes
- This violates the priority model
 - High priority tasks can only be prevented from running by higher priority tasks
- Consequences:
 - Increased waiting times for higher-priority processes
 - Unpredictable behavior in systems with real-time constraints
 - Overall performance and responsiveness degradation

Priority inversion

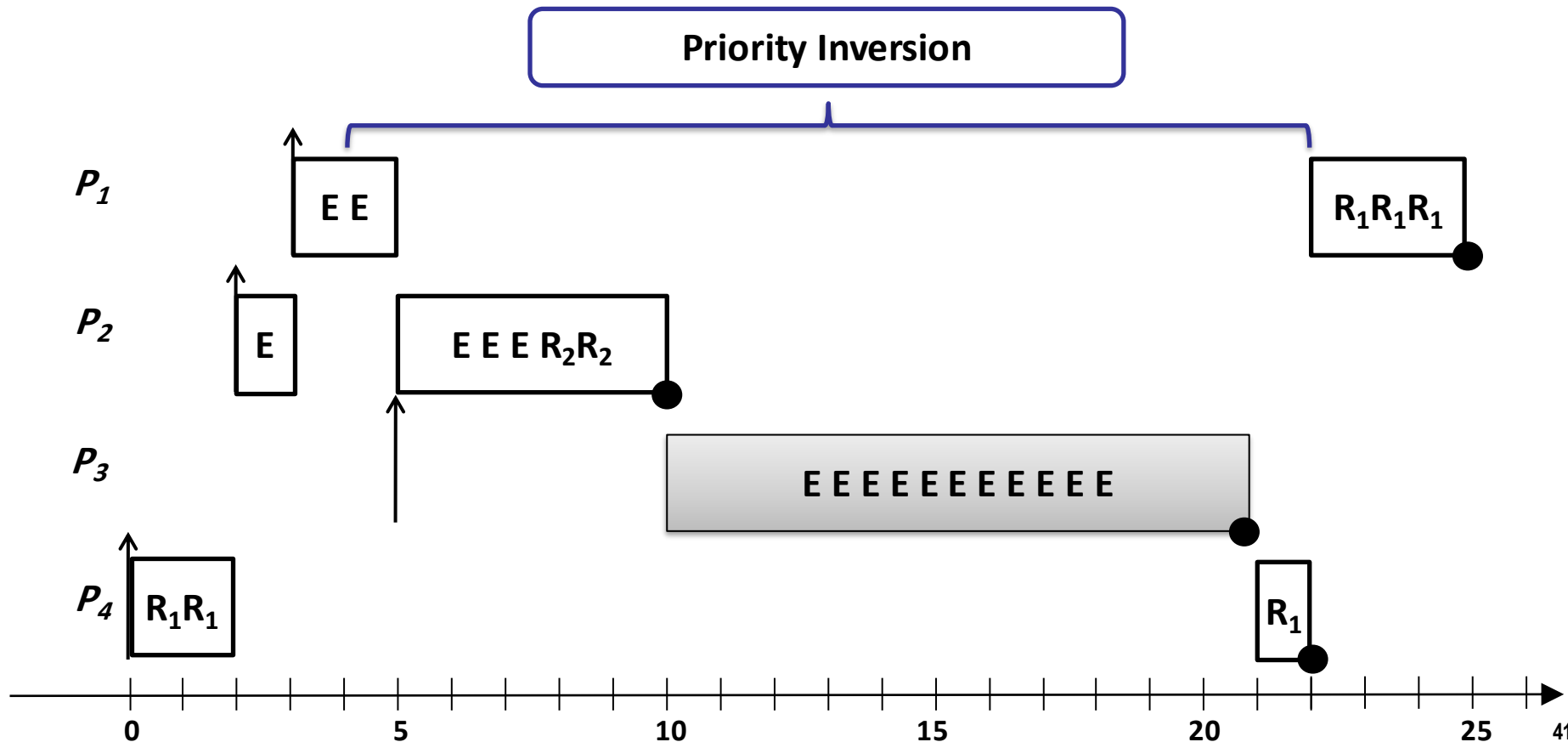
Process	Execution time	Priority	Arrival instant	Execution sequence
P_1	5	1	3	E E $R_1 R_1 R_1$
P_2	5	2	2	E E E E $R_2 R_2$
P_3	7	3	0	$R_1 R_1 R_1$ E $R_2 R_2$ E

A higher priority process (P_1) waits for a lower priority process (P_3) while middle priority processes are allowed to execute



Priority inversion

- If no rule is applied when sharing resources, it is impossible to determine the **maximum blocking time**
 - A widely known example: Mars Pathfinder (1997)



Avoiding priority inversion

■ OSeS mitigate priority inversion by using dynamic adjustments

- Ensure low-priority processes release shared resources quickly
- Prevent blocking of higher-priority processes

■ Random boosting (Windows)

- The scheduler periodically or opportunistically raises the priority of waiting processes
- Helps break priority inversion by giving blocked or low-priority processes a chance to run
- Simple and low-overhead, but non-deterministic and not guaranteed to resolve all cases

Avoiding priority inversion

■ Priority Inheritance Protocol (Linux with PREEMPT_RT patch)

- A low-priority process holding a resource temporarily inherits the higher priority of the blocked process
- Keeps the elevated priority until the resource is released
- Widely used, but can lead to chained inheritance (nested dependencies)

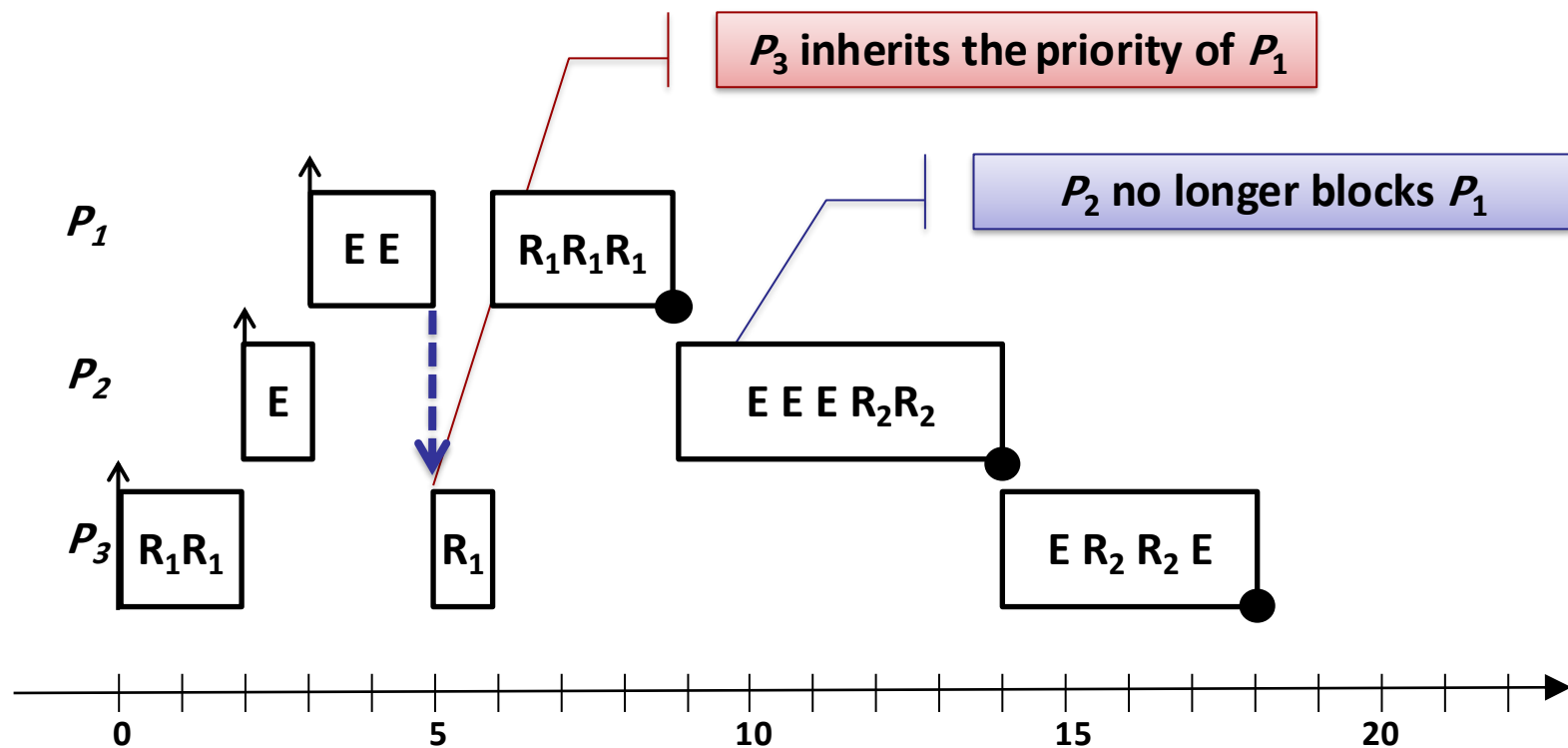
■ Priority Ceiling Protocol (Linux with PREEMPT_RT patch)

- Each resource is assigned a priority ceiling, equal to the highest priority of any process that may use it
- A process holding the resource executes at the ceiling priority
- Prevents priority inversion proactively
- More complex, requires static analysis of resource usage

Priority Inheritance Protocol

- When a higher-priority process blocks, **the lower-priority process using the resource “inherits” the higher priority** until it unlocks the resource
- + Allows several blocking periods but guarantees a maximum blocking period
- Significantly increases the overhead of the scheduler
 - Each process must donate its priority to all processes it is blocked on
 - (This is why the Mars Pathfinder was sent to Mars with priority donation turned off...)

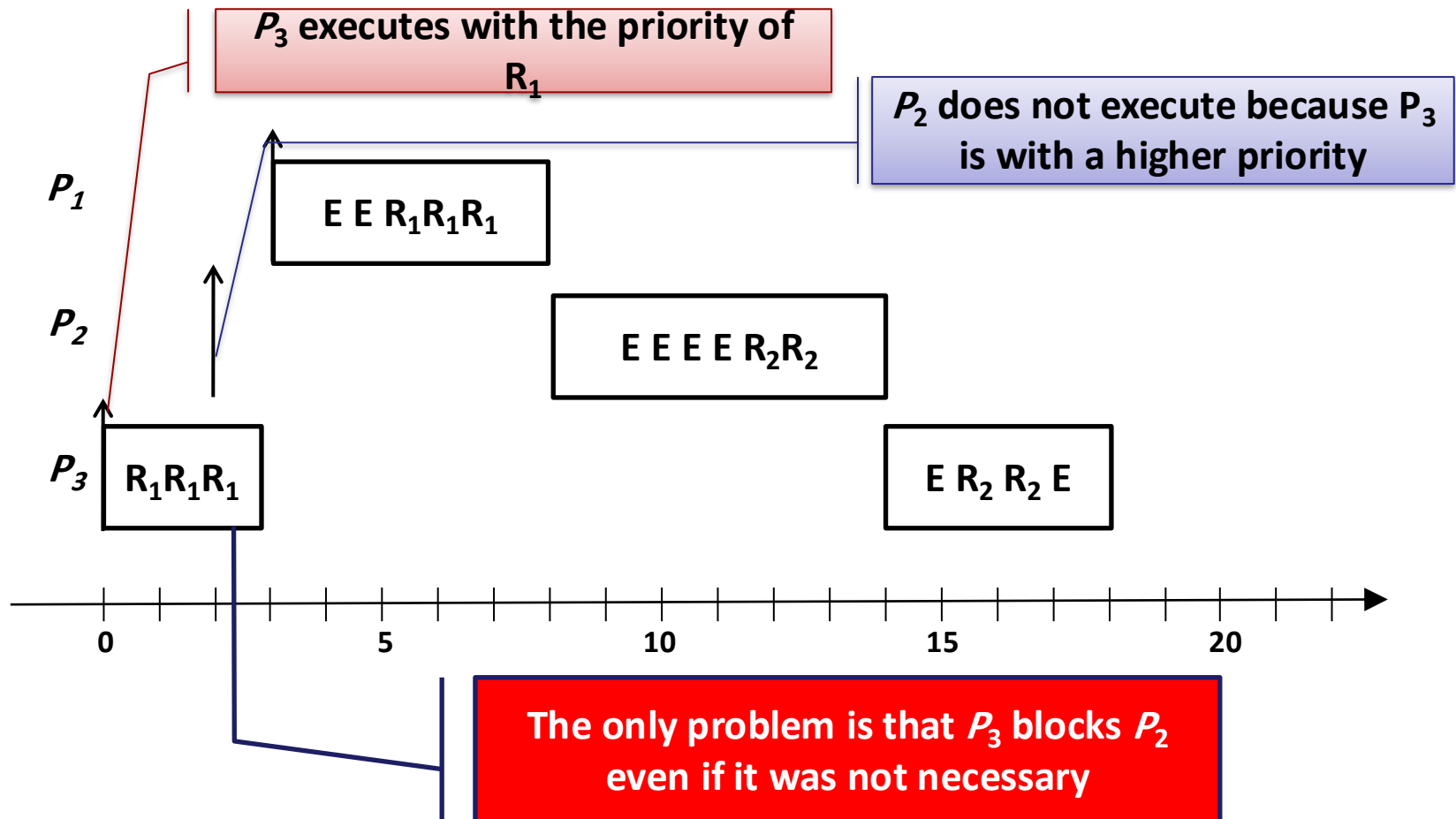
Priority Inheritance Protocol



Priority Ceiling Protocol

- Resources are given a priority which is equal to the **highest priority** of the processes that use the resource (ceiling)
- When holding the resource, processes execute with the **priority of the resource** (ceiling)
 - Until it unlocks the resource
- + Allows only one blocking period
 - Also prevents deadlocks
- Introduces unnecessary blocking

Priority Ceiling Protocol



Conclusions

- **Fixed priority scheduling is a widely used approach for its simplicity and deterministic behavior**
- **Advantages**
 - Predictable task execution timing
 - Straightforward implementation and analysis
 - Well-suited for systems with strict timing requirements
- **Potential problems**
 - Can lead to starvation, compromising system fairness and lead to poor performance for certain tasks
 - Can lead to priority inversion scenarios when sharing resources, compromising system responsiveness